GRAPH-BASED PREDICTIVE AGENT FOR DECISION MAKING IN DYNAMIC ENVIRONMENTS

CS5491 PROJECT MILESTONE REPORT (MARCH 2026)

Shutang Zhang, Jie Liang & Kehong Pan
Department of Computer Science, City University of Hong Kong
{szhang2569-c, jliang443-c, khpan3-c}@my.cityu.edu.hk

1 PROBLEM DESCRIPTION AND MOTIVATION

Efficient navigation in dynamic environments remains a fundamental challenge in reinforcement learning (RL). Moving obstacles induce high-dimensional and non-stationary state distributions, where an agent must avoid collisions while making progress toward a goal. Many standard RL pipelines treat observations as unstructured vectors, which often weakens relational reasoning (agent–goal–obstacle geometry) and leads to reactive behaviors that respond only to immediate threats.

This project studies whether two inductive biases can improve proactive navigation: (1) **graph-based relational encoding** via a Graph Attention Network (GAT), which yields a permutation-invariant latent representation and supports structural generalization across varying numbers of obstacles; and (2) **short-horizon latent prediction** via a GRU-based world model, which enables the agent to anticipate future relational configurations before committing to an action.

Milestone focus. The primary goal of this milestone is to demonstrate that we have built a complete and reproducible experimental pipeline: environment, baselines, training, evaluation, reporting, and tests. While the current learned-policy results are not yet fully optimized, we provide preliminary numbers, identify why some results are unreasonable or unstable, and outline concrete next steps for improvement.

2 METHOD DESIGN

2.1 ARCHITECTURE OVERVIEW

Our approach integrates three modules trained end-to-end with Proximal Policy Optimization (PPO):

- **Relational Encoder (GAT).** The environment at time t is represented as a dynamic graph $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t)$ whose nodes include the agent, goal, and moving obstacles. A GAT-style encoder maps the graph to a latent state z_t that is permutation-invariant to entity ordering and robust to varying obstacle counts.
- **Predictive World Model (GRU in latent space).** A GRU-based transition model learns $z_{t+1} = f_\theta(z_t, a_t)$. Instead of directly regressing obstacle coordinates, it forecasts the evolution of the relational latent field. We use a short predictive horizon H (default $H = 3$).
- **Policy Optimization (PPO actor–critic).** The policy is conditioned on both the current and predicted future states:
$$\pi(a_t \mid z_t, \{z_{t+k}\}_{k=1}^{H}).$$
An auxiliary prediction loss is added to the PPO objective to jointly train the predictive module.

2.2 BASELINES IMPLEMENTED

To isolate the effect of structural encoding and forecasting, we implemented:

- **MLP-PPO:** PPO with flattened vector observations (no relational prior).
- **GNN-PPO:** GAT encoder without the predictive rollout module (relational-only ablation).

- **Oracle (A*):** A time-expanded A* planner with privileged access to obstacle motion (upper-bound reference for path efficiency).

3 IMPLEMENTATION PROGRESS (MILESTONE DELIVERABLES)

At the milestone stage, we have completed the following components.

3.1 ENVIRONMENT AND BENCHMARK SUITE

- **Dynamic Arena:** Implemented a custom `Gymnasium`-style navigation arena with moving obstacles, including constant-velocity and Brownian-motion dynamics, categorized into three difficulty tiers (`easy/medium/hard`).
- **Frozen Evaluation Suite:** To guarantee strict reproducibility across baselines, we systematically generated a frozen evaluation dataset using `generate_eval_suite.py`. By fixing the `base-seed` to 2026, we sampled 100 distinct episodes per difficulty tier (300 in total) and serialized the environment configurations to `configs/eval_suite.json`. All evaluated methods in this report are benchmarked on this exact set of test instances.

3.2 TRAINING, EVALUATION, AND REPORTING PIPELINE

- Unified entrypoints for training and evaluation (`train.py` and `eval.py`).
- A centralized experiment matrix (`configs/experiments.json`) that declares all main experiments, ablations, and stability tests.
- Automated result aggregation and asset generation scripts (`aggregate_results.py`, `make_figures.py`, `generate_report_assets.py`) to produce end-to-end tables and plots from raw logs.

3.3 TESTING AND REPRODUCIBILITY

- Created a robust testing suite (`tests/`) with 13 smoke tests covering environment resets, the oracle planner, neural network forward passes, and the complete training loop. All tests currently pass, verifying pipeline stability.

4 EXPERIMENTAL SETUP

4.1 METRICS

We compute:

- **Success Rate:** fraction of episodes reaching the goal without collision.
- **Collision Rate:** fraction of episodes ending in collision.
- **Path Efficiency:** ratio of the agent path length to the oracle optimal path length, computed on successful episodes only.
- **Average Episode Length:** mean number of steps per episode.

4.2 TRAINING CONFIGURATION (CURRENT DEFAULT)

Unless otherwise specified, PPO hyperparameters are: `rollout_steps=256`, `update_epochs=4`, `minibatch_size=64`, `lr=3e-4`, `clip_coef=0.2`. Training budgets are scaled by difficulty: easy (30k–50k steps), medium (50k–80k), hard (70k–100k).

4.3 OPERATING ENVIRONMENT

All milestone experiments and pipeline testing were executed locally on a machine equipped with an **AMD Ryzen R7-5800H CPU** and an **NVIDIA RTX 3060 Laptop GPU (6GB VRAM)**. The

software environment was managed via **Conda 24.11.3** running **Python 3.12.12**. Key package dependencies include `torch >= 2.1` and `torch-geometric >= 2.5` for the neural components, `gymnasium >= 0.29` for the RL environment API, and standard data science tools including `numpy >= 1.24`, `pandas >= 2.0`, `matplotlib >= 3.8`, and `pytest >= 8.0`.

5 Preliminary Results (Pipeline Validation Run)

Given the intensive computational workload and the tight timeline of this milestone, we restricted our initial execution to the **10 primary experiments** designated in the `main` group of our configuration file. The extensive ablations (e.g., $H = 1$) and multi-seed stability runs (seeds 1 and 2) are fully configured in the pipeline but deferred to the final phase.

Table 1 summarizes the preliminary outcomes. We emphasize that these runs serve primarily as a **pipeline validation baseline** rather than a fully tuned final comparison (see Section 6).

Model	Difficulty	Success ↑	Collision ↓	Path Eff. ↓	Avg Len.
oracle	Easy	1.000	0.000	1.000	5.580
mlp	Easy	0.080	0.390	1.000	20.050
gnn	Easy	0.140	0.350	1.000	18.850
predictive (H=3)	Easy	0.140	0.480	0.883	18.510
oracle	Medium	1.000	0.000	1.000	6.460
mlp	Medium	0.120	0.530	1.000	23.230
gnn	Medium	0.090	0.580	1.000	22.580
predictive (H=3)	Medium	0.030	0.520	1.000	26.160
oracle	Hard	1.000	0.000	1.000	7.860
mlp	Hard	0.020	0.730	1.000	25.540
gnn	Hard	0.020	0.730	1.000	25.540
predictive (H=3)	Hard	0.070	0.620	1.013	27.240

Table 1: Main preliminary results across difficulty tiers. Oracle (A*) is a privileged upper-bound reference. All RL models were trained with seed=0.

Visual summary of metrics. To complement Table 1, we provide bar-chart visualizations of success rate, collision rate, and path efficiency across difficulty tiers (Figures 1–3). These plots make relative trends among methods easier to inspect, especially under low-success regimes.

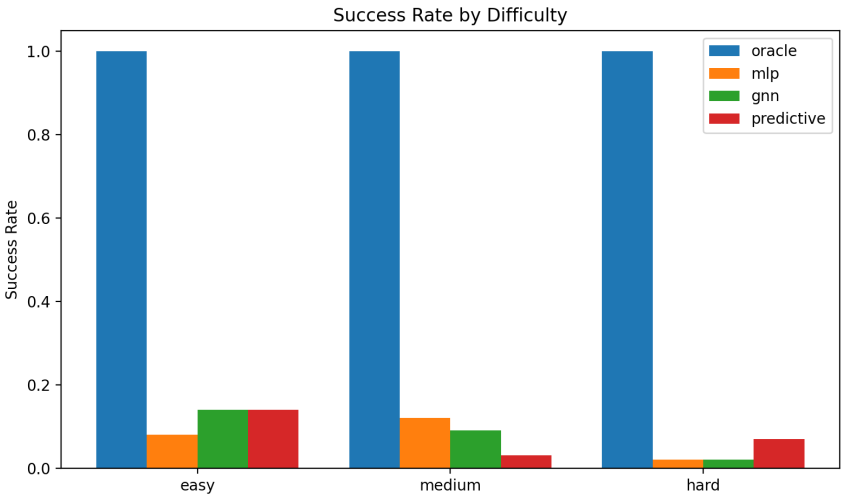

Figure 1: Success rate by difficulty for Oracle, MLP-PPO, GNN-PPO, and Predictive-PPO (H=3).

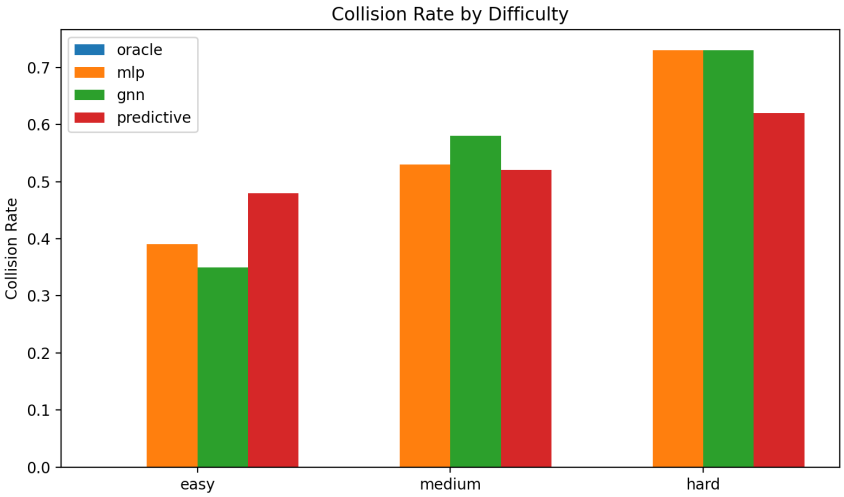

Figure 2: Collision rate by difficulty. Lower is better.

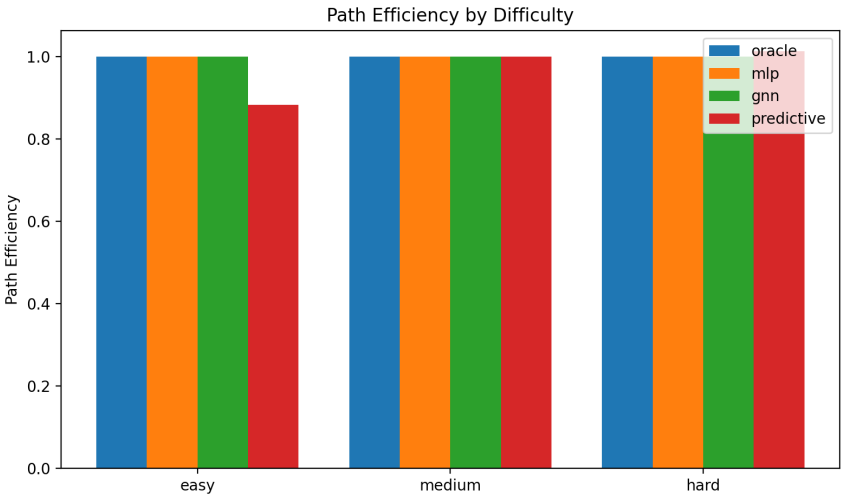

Figure 3: Path efficiency by difficulty, normalized by the oracle path length on successful episodes.

Training dynamics. Figure 4 shows the training return trajectories for the nine learned-policy runs in the main group (MLP/GNN/Predictive across easy/medium/hard). The curves are intentionally reported as raw trajectories to expose optimization variance and support transparent diagnosis of instability in this milestone phase.

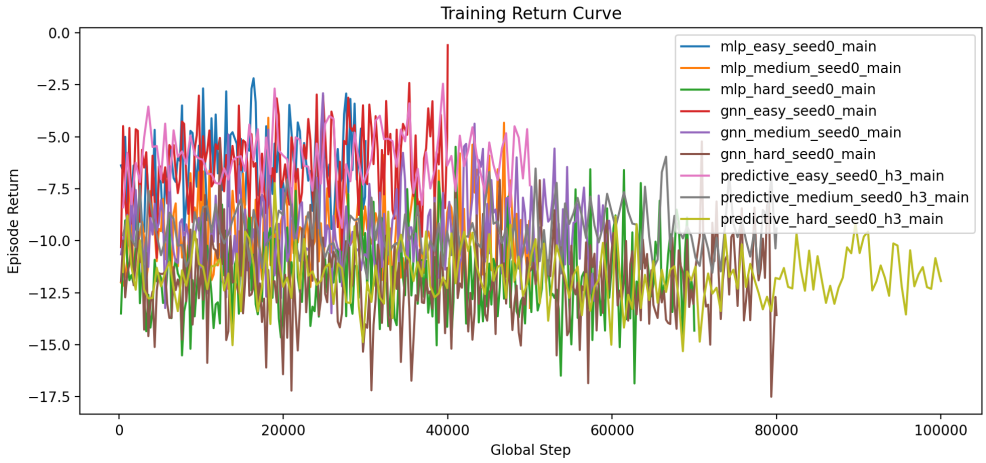

Figure 4: Training return curves (episode return vs. global step) for all learned models in the main experimental group.

5.1 Key Observations

- **Easy:** GNN-PPO and Predictive-PPO both improve success rate over MLP-PPO, suggesting that relational encoding provides immediate benefits. However, the predictive module does not yet increase success rate in this tier.
- **Medium:** All learned models have high collision rates. Predictive-PPO underperforms in success rate in this preliminary run, indicating that forecasting may be harder to learn under the current training budget and hyperparameters.
- **Hard:** Predictive-PPO achieves higher success rate and lower collision rate than MLP/GNN in this seed=0 run, providing an early signal that short-horizon prediction can help under high obstacle density.

6 Why Some Results Look Unreasonable (and How We Will Fix Them)

The current results are **not yet tuned**, and several factors make the preliminary comparisons appear unstable or counter-intuitive:

(1) No systematic hyperparameter tuning yet. Our PPO settings, prediction loss coefficient, and horizon were chosen as sensible defaults. The predictive agent introduces additional optimization difficulty (jointly learning policy/value and a latent transition model), which requires careful learning-rate and auxiliary-loss balancing.

(2) Limited training budgets and few random seeds. The preliminary table reports a single seed (seed=0). With small success rates (especially on hard), single-seed outcomes can be dominated by stochastic variance. In the final phase, we will execute the pending `stability` runs.

(3) Metric interpretability when success is rare. Path efficiency is computed on successful episodes only. When success rate is low, this metric may be based on a small number of episodes and thus becomes noisy. We will include additional diagnostics (e.g., near-miss rate or minimum distance to obstacles) moving forward.

7 PLANNED NEXT STEPS (BEFORE FINAL REPORT)

- **Ablations:** Run the queued $H = 1$ vs. $H = 3$ experiments to isolate the forecasting contribution.
- **Stability:** Execute the multi-seed configurations (seeds 1 and 2) mapped in `experiments.json`.
- **Longer training:** Increase `total_timesteps` for medium/hard to test whether predictive advantages grow with more experience.
- **Qualitative analysis:** Leverage our `export_rollouts.py` script to visualize failure modes.

8 CODEBASE STRUCTURE

Our submitted codebase is modularized to ensure reproducibility. The core hierarchical structure is organized as follows:

- `configs/` — Locked parameters and dataset initialization.
  - `eval_suite.json`: The frozen evaluation suite (base seed 2026).
  - `experiments.json`: The formal experiment matrix defining the 10 main runs, ablations, and stability tests.
- `dynamic_nav/` — The core library package.
  - `models/`: Neural architecture implementations (`mlp.py`, `gnn.py`, `predictive.py`).
  - `env.py` & `oracle.py`: Environment definitions and the A* pathfinding upper-bound.
  - `ppo.py`: Proximal Policy Optimization trainer logic.
- **Execution Scripts (Root)** — Entrypoints for the pipeline.
  - `generate_eval_suite.py`: Script generating the reproducible benchmark suite.
  - `train.py` & `eval.py`: Unified individual training and evaluation entries.
  - `run_experiments.py`: Batch executor parsing `experiments.json`.
  - `aggregate_results.py` & `make_figures.py`: Results compilation and visualization.
- `tests/` — 13 unit and integration tests passing successfully via PyTest.
- `outputs/` & `checkpoints/` — Directories tracking artifacts (logs, weights, figures).